

第十五讲 SLAM

王 宇

中科院自动化所

章节安排

一

什么是SLAM

二

SLAM发展简史

三

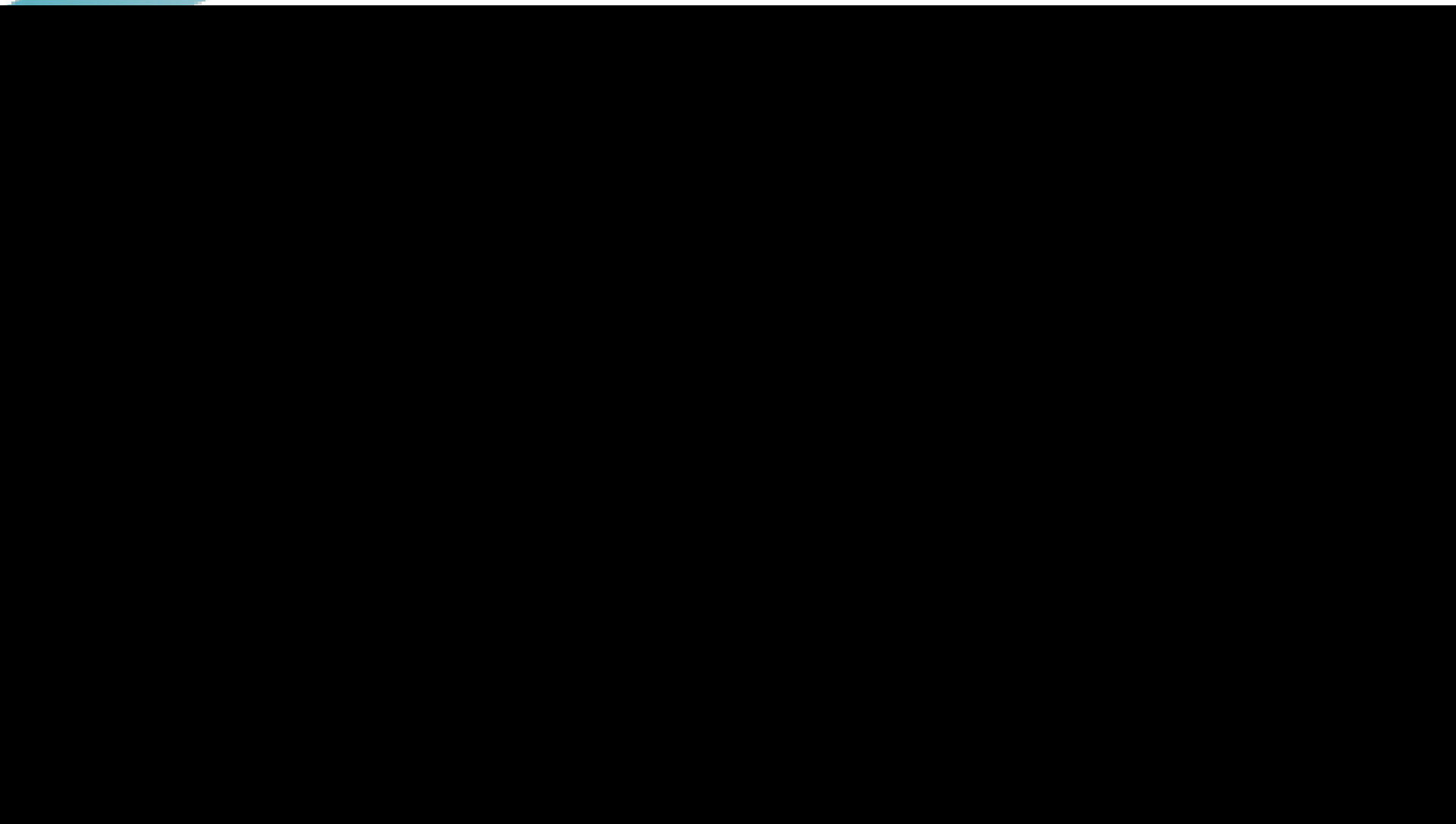
视觉SLAM算法

四

SLAM技术的基础理论

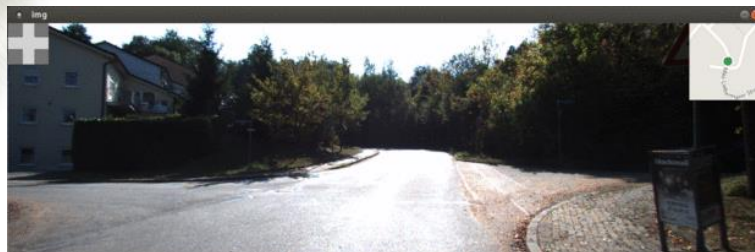
五

ROS下的SLAM实现



什么是SLAM

- 计算机视觉



物体识别
物体跟踪
物体检测
语义分割

.....
SLAM

现实世界中的相机

视频序列

计算机视觉任务

什么是SLAM

SLAM是什么？



Universidad
Zaragoza



Instituto Universitario de Investigación
en Ingeniería de Aragón
Universidad Zaragoza

ORB-SLAM2: an Open-Source SLAM System
for Monocular, Stereo and RGB-D Cameras

Raúl Mur-Artal and Juan D. Tardós

raulmur@unizar.es

tardos@unizar.es

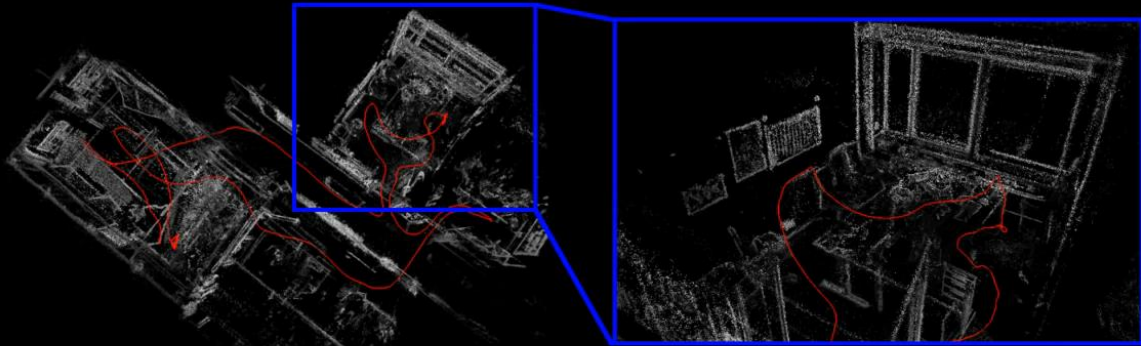
室内/室外定位

什么是SLAM

SLAM是什么?

稀疏-半稠密重建

Direct Sparse Odometry
Jakob Engel,^{1,2} Vladlen Koltun,² Daniel Cremers¹
July 2016



TUM¹Computer Vision Group
Technical University Munich

²Intel Labs 

什么是SLAM

SLAM是什么?

ElasticFusion: Dense SLAM Without A Pose Graph

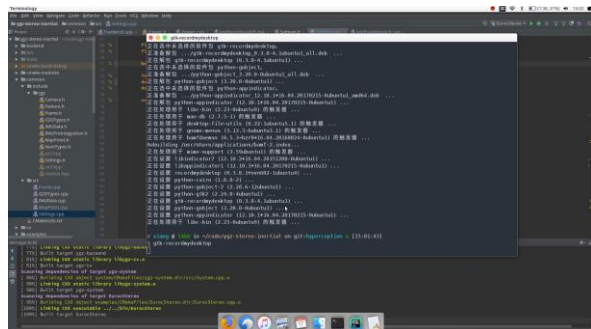
Thomas Whelan, Stefan Leutenegger, Renato Salas-Moreno, Ben Glocker, Andrew Davison

Imperial College London

稠密重建

什么是SLAM

- SLAM可以用在哪些地方?



手持设备定位



自动驾驶定位

De-noising, Stabilizing and Completing 3D Reconstructions On-the-go using Plane Priors

Maksym Dzitsiuk, Jürgen Sturm, Robert Maier, Lingni Ma, Daniel Cremers



Technical University of Munich,
Google



AR (增强现实)

什么是SLAM

- **自主运动的两大基本问题**
 - 我在什么地方？——**定位**
 - 周围环境是什么样子？——**建图**
 - 定位与建图=内外兼修 定位侧重对自身的了解，建图侧重对外在的了解
- **相互关联**
 - 准确的定位需要精确的地图
 - 精确的地图来自准确的定位



什么是SLAM

- **传感器**

- 机器人感知外界环境的手段
- 种类：内质的/外质的
 - 内质：感受机器人本体信息
 - IMU、激光、相机
 - 外质：安装于环境中的
 - 二维码Marker、GPS、导轨、



(a)



(b)



(c)



(d)



(e)



(f)

- **环境限制了传感器的形式**

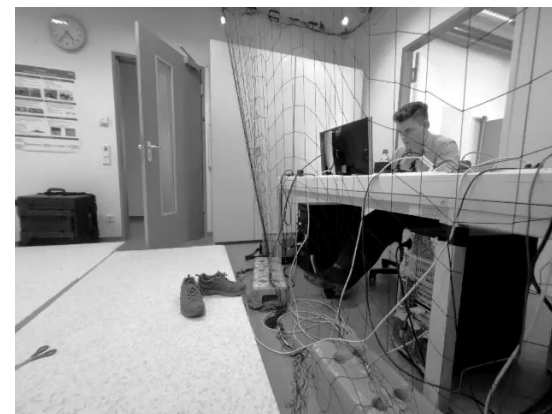
- GPS：需要能接收到卫星信号的环境
- Marker、导轨：需要环境允许安装
- 相比之下，激光、相机等携带式传感器更加自由

什么是SLAM

- **相机**
 - 以一定速率采集图像、形成视频
- **分类**
 - 单目相机 Monocular
 - 双目相机（立体相机） Stereo
 - 深度相机 RGB-D
 - 其他 全景、Event Camera
- **相机的特点**
 - 以二维投影形式记录了三维世界的信息
 - 该过程丢掉了一个维度：距离



图 2-3 形形色色的相机：单目，双目和深度相机。



什么是SLAM

• 各类相机的区别

- 单目：无深度 深度需要其他手段估计
- 双目：通过视差计算深度
- RGB-D：通过物理方法测量深度

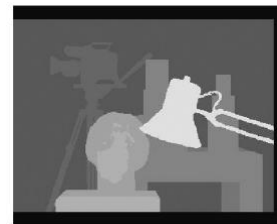
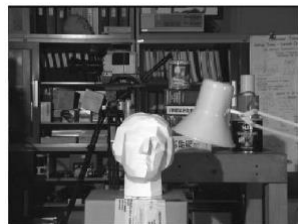
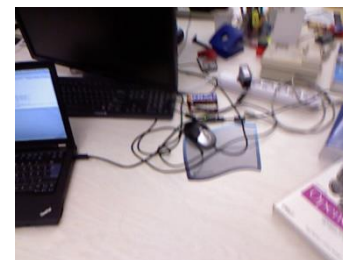
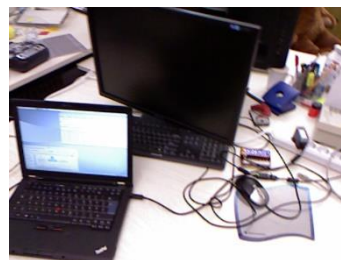


图 2-4 单目视觉中的尴尬：不知道深度时，手掌上的人是真人还是模型？



当相机运动时，可通过视差确定深度

深度即三维信息，对SLAM来说至关重要

什么是SLAM

- **视觉SLAM框架**
 - 前端: Visual Odometry
 - 后端: Optimization
 - 回环: Loop Closing
 - 建图: Mapping

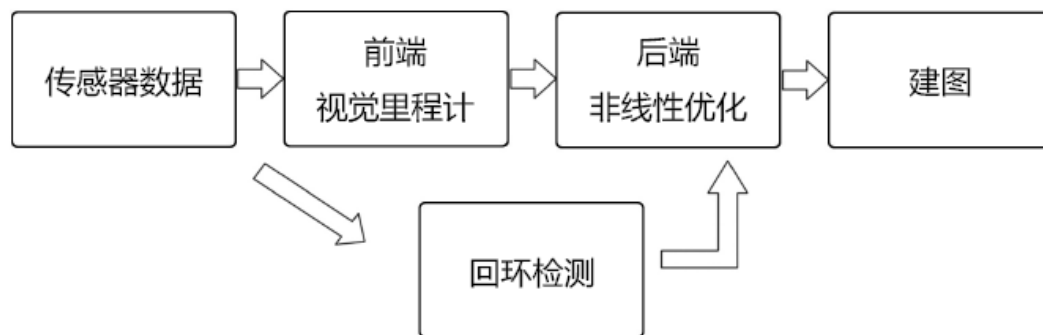


图 2-7 整体视觉 SLAM 流程图。

什么是SLAM

- **视觉里程计**

- 估计邻近时刻的相机运动
- 最简化：两个图像的相对运动

- **方法**

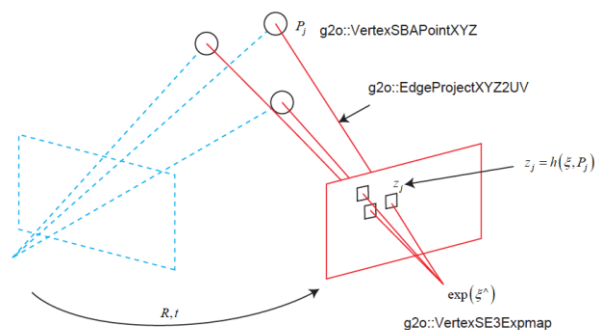
- 特征点法
- 直接法

- **后端**

- 从带有噪声的数据中估计最优轨迹与地图
- 最大后验概率估计
- 滤波器
- 图优化



图 2-8 相机拍摄到的图片与人眼反应的运动方向。



什么是SLAM

回环检测

- 检测相机是否到达过之前位置
- 判断与之前位置的差异
- 计算图像间相似性
- 词袋模型

建图

- 导航、规划、通讯、交互、可视化
- 度量地图、拓扑地图
- 稀疏地图、稠密地图

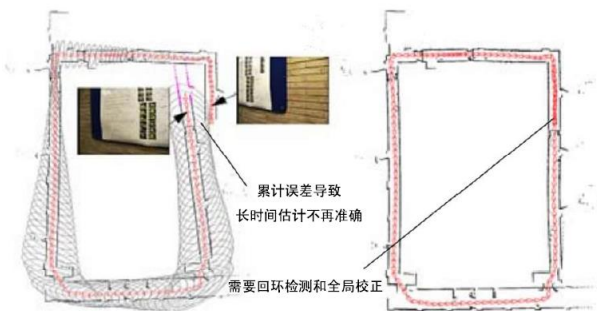


图 2-9 累计误差与回环检测的校正结果 [10]。

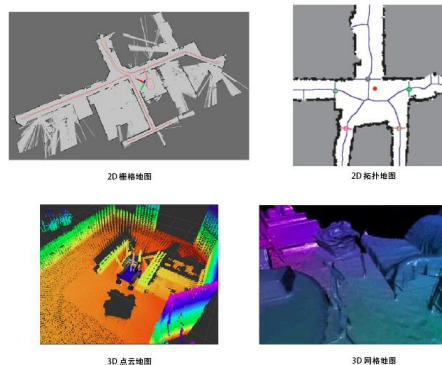


图 2-10 形形色色的地图：2D 栅格地图、拓扑地图以及 3D 点云地图和网格地图 [12]。

视觉SLAM的数学描述

- SLAM问题的数学描述

$$\begin{array}{l} \text{运动方程} \\ \text{观测方程} \end{array} \quad \left\{ \begin{array}{l} \mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k, \mathbf{w}_k) \\ \mathbf{z}_{k,j} = h(\mathbf{y}_j, \mathbf{x}_k, \mathbf{v}_{k,j}) \end{array} \right.$$

问题:

- 位置是三维的, 如何表述?

参考: 三维空间的刚体运动

点与坐标系

旋转矩阵

旋转向量和欧拉角

四元数

- 观测模型如何表述? 相机模型
- 已知 $\mathbf{u}, \mathbf{z}$ 时, 如何推断 $\mathbf{x}, \mathbf{y}$?

章节安排

一

什么是SLAM

二

SLAM发展简史

三

视觉SLAM算法

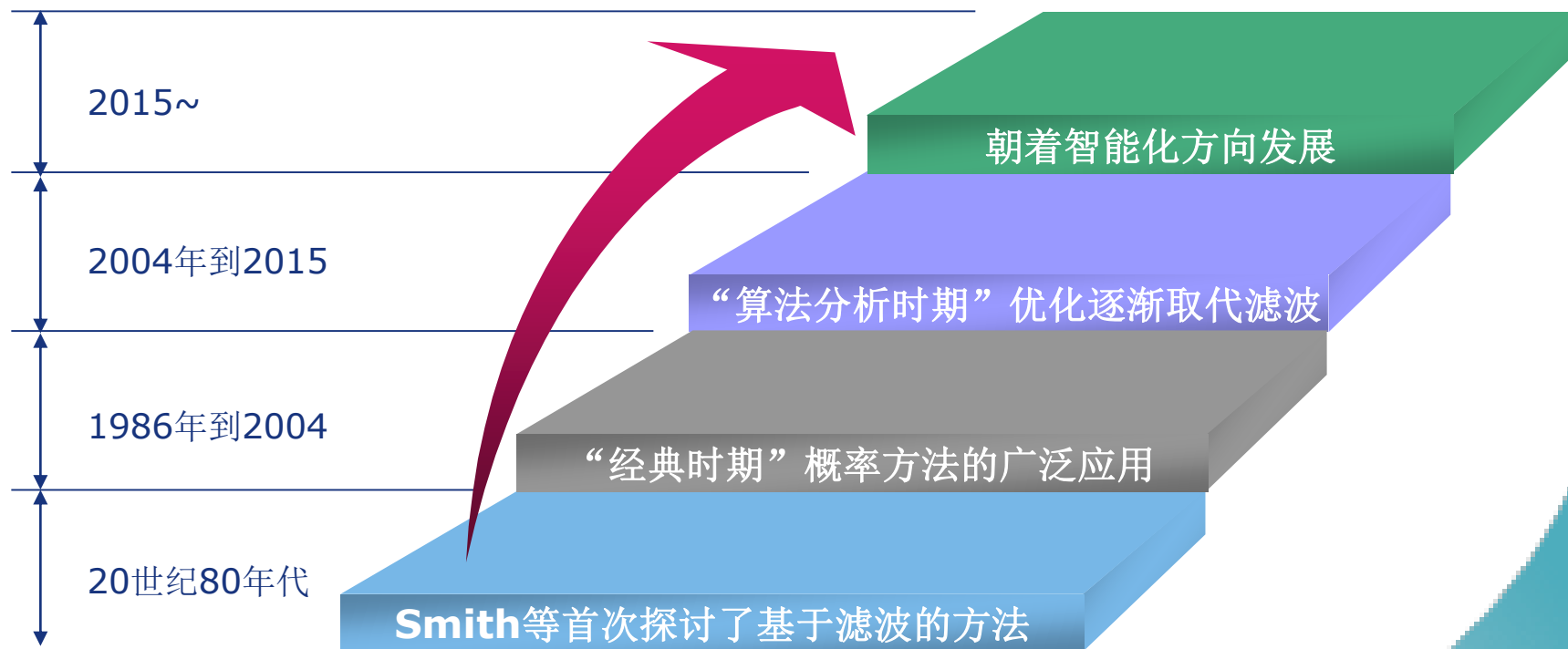
四

SLAM技术的基础理论

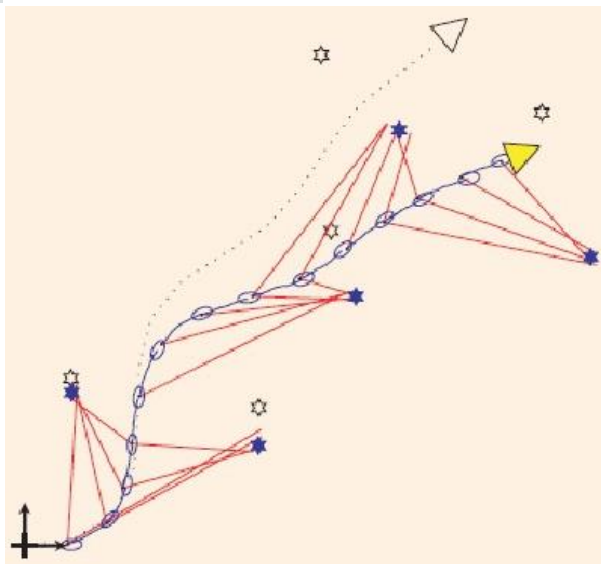
五

ROS下的SLAM实现

SLAM发展简史



SLAM发展简史



所谓滤波思想是指将SLAM建模为一个状态估计问题，当时被称为“空间状态不确定性的估计问题”。

$$\begin{cases} x_k = f(x_{k-1}, u_k) + q_k & \text{运动方程} \\ z_{k,j} = h(y_j, x_k) + p_{k,j} & \text{观测方程} \end{cases}$$

滤波法SLAM就是根据运动输入 u 和观测 z ，求使得下式所示条件概率分布最大的状态

$$x_{Map}^* = \arg \max P(x | z, u)$$

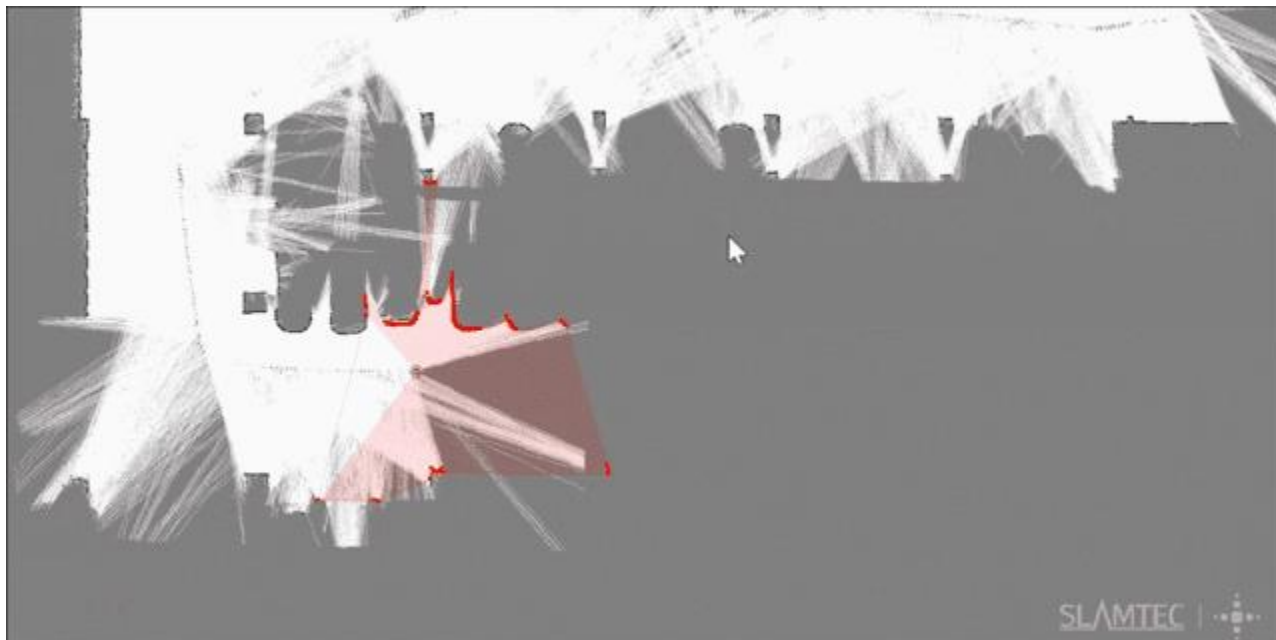
SLAM发展简史



2004年到2015年期间，SLAM问题的研究进入“算法分析时期”，Dissanayake等人从状态的可观测性、状态估计的收敛性、一致性和算法计算效率等角度对这一时期的部分工作做了综述，Cesar等人对SLAM的研究做了系统性的总结。

与此同时基于优化的SLAM技术逐渐成熟。

SLAM发展简史

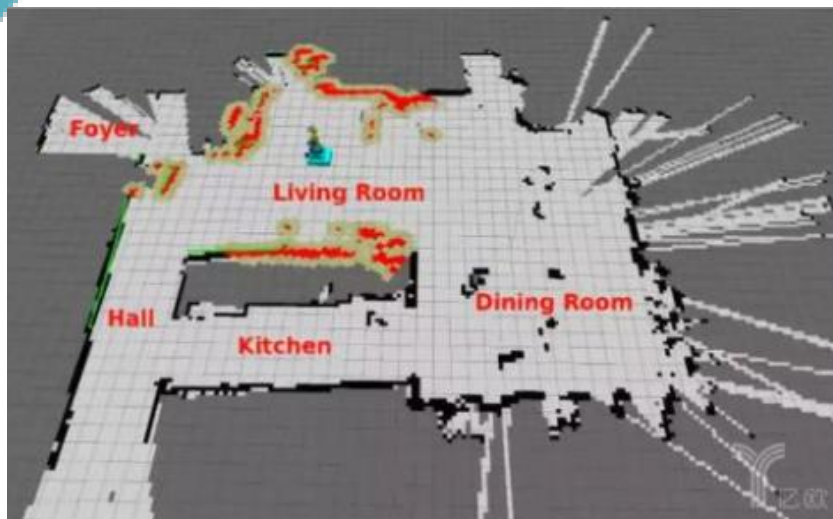


传感技术和算法的进步驱动滤波类方法发展为优化类方法

滤波类方法：只考虑相邻时刻间系统状态的约束

优化类方法：将所有的历史状态都视为可优化的变量。减少全局误差

SLAM发展简史



基于优化的SLAM技术框架在2015前后基本走向了成熟。之后，随着深度学习等智能技术的发展，与SLAM技术结合生成环境的语义地图，构建环境的语义知识库，有望在很大程度上提升机器人的智能水平。

SLAM发展简史

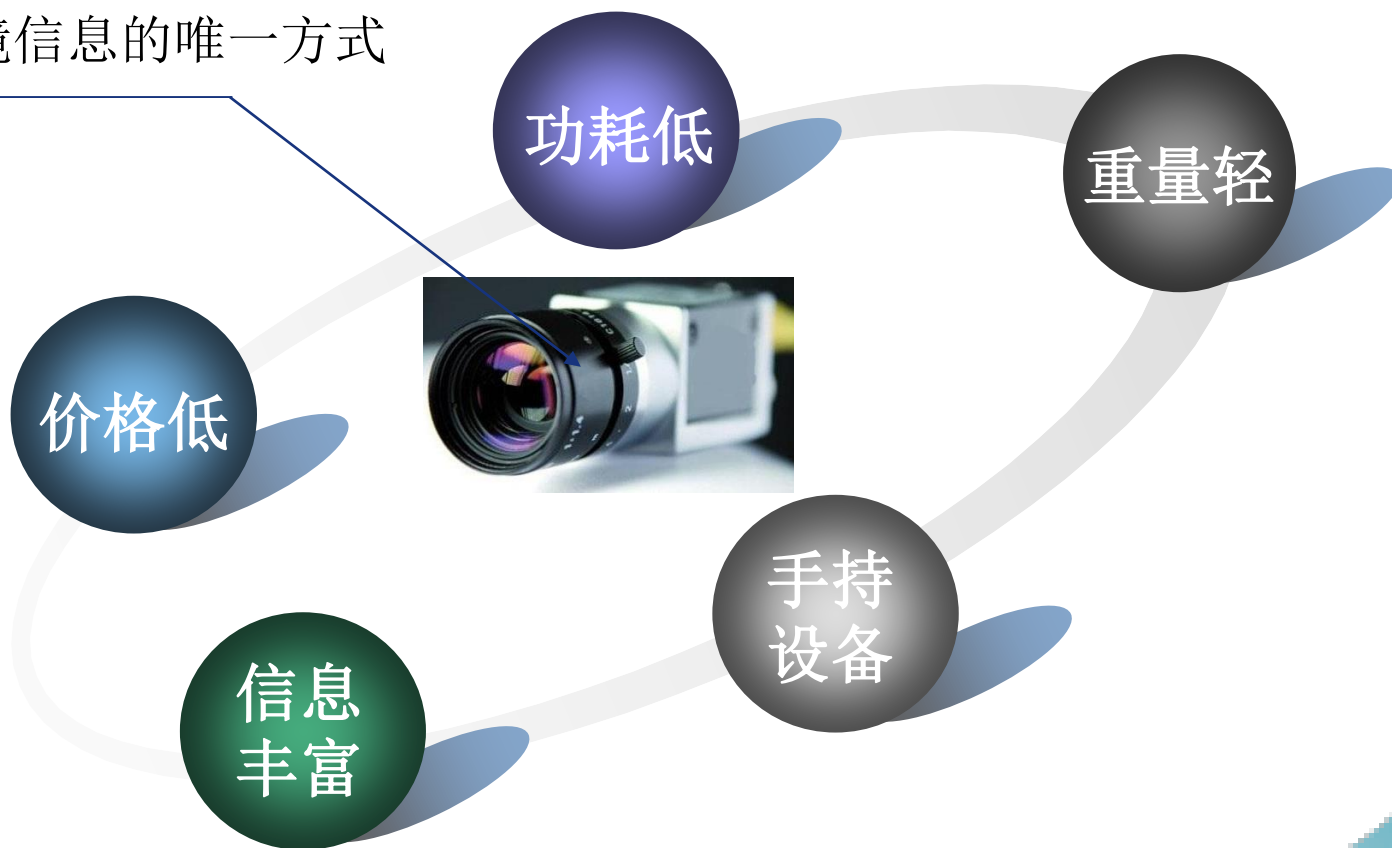


章节安排

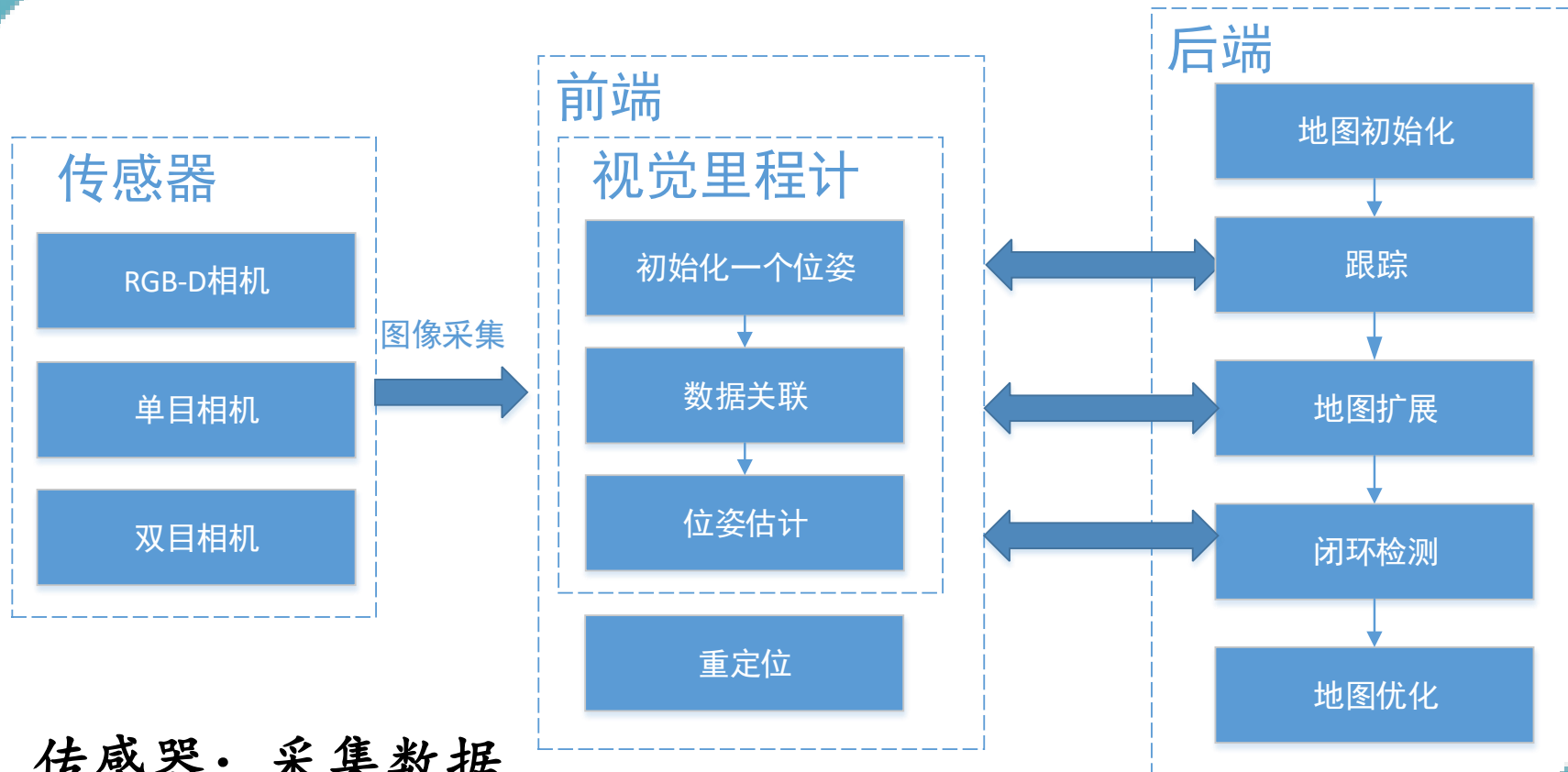
- 一 什么是SLAM
- 二 SLAM发展简史
- 三 视觉SLAM算法
- 四 SLAM技术的基础理论
- 五 ROS下的SLAM实现

视觉SLAM框架

视觉SLAM以相机作为获取环境信息的唯一方式



视觉SLAM框架



传感器：采集数据

前端：处理局部信息，主要负责定位

后端：全局性的优化，主要负责建图

视觉SLAM框架

传感器

RGB-D相机

单目相机



图像

前端

视觉里程计

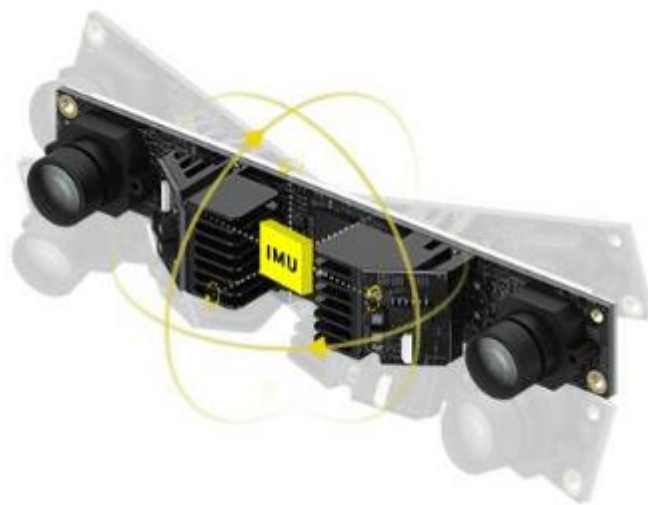


化

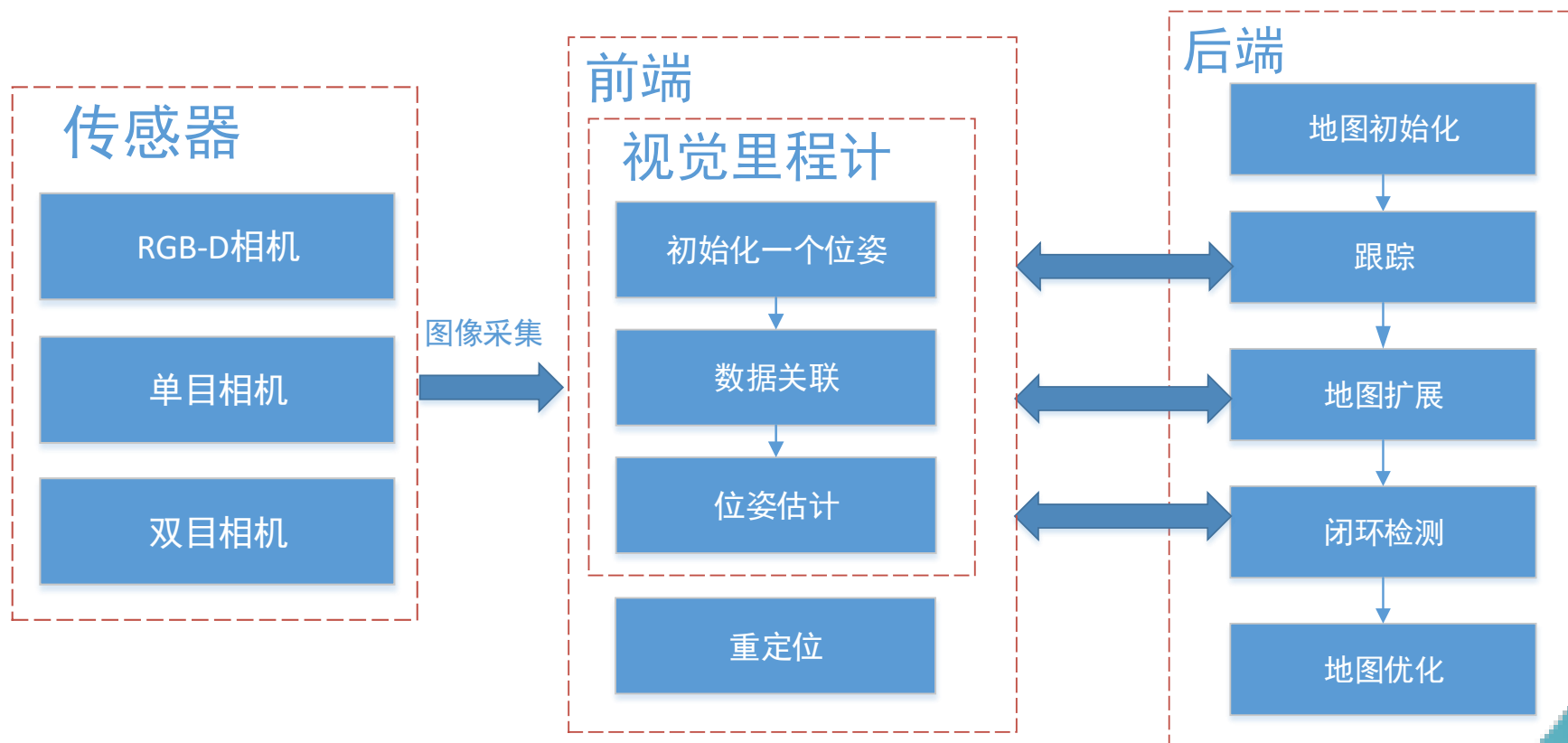
展

检测

优化



视觉SLAM框架



视觉SLAM框架

视觉里程计

初始化— $\mathbf{P}_0$ 位姿

$\mathbf{P}_{0:N} = \{\mathbf{P}_0, \dots, \mathbf{P}_N\}$ 数据关联

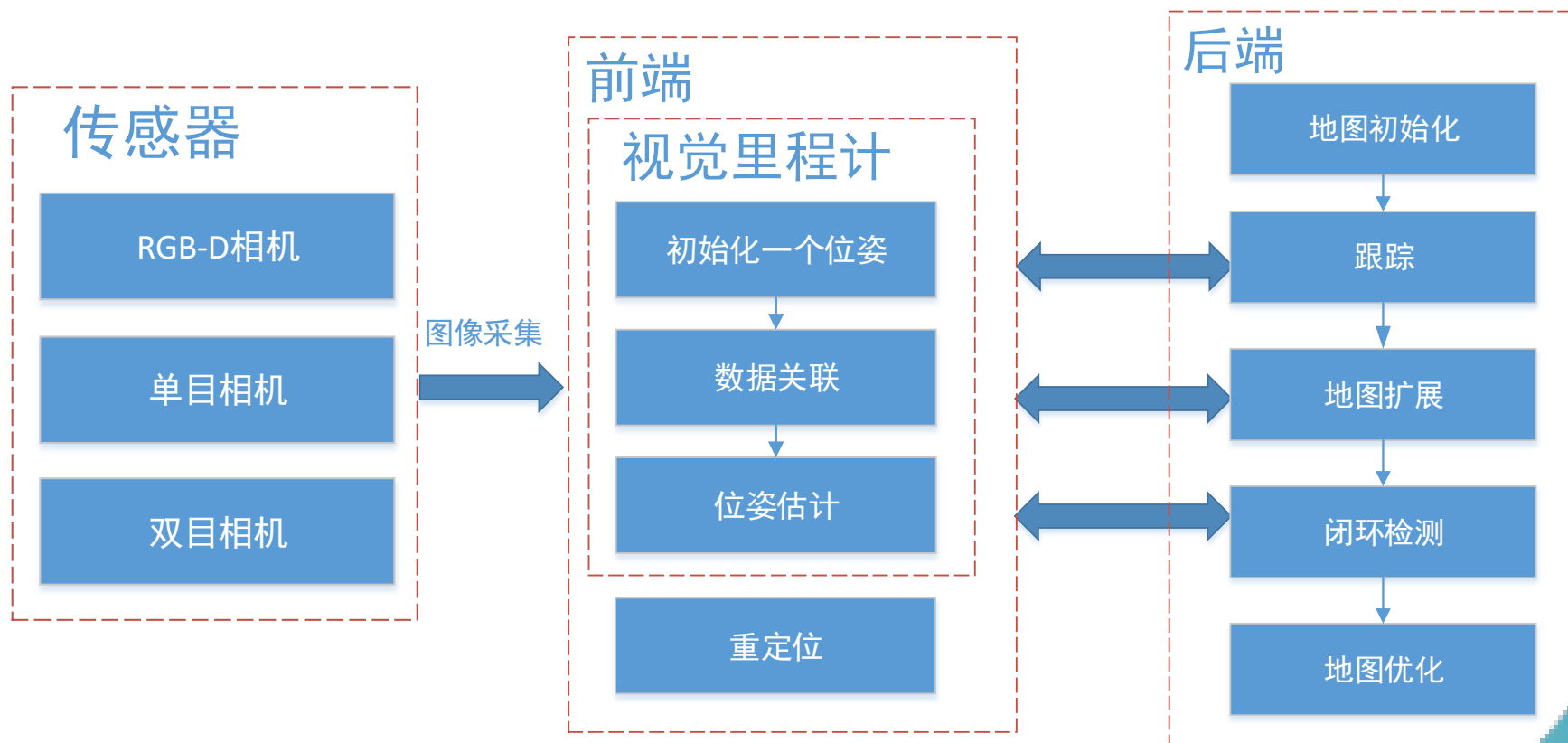
位姿估计
 $\mathbf{P}_k = \mathbf{T}_k \mathbf{P}_{k-1}$

重定位

特征点法：根据图像中的特征匹配关系得到相邻帧间的相机运动估计
直接法：利用像素点的光度误差建立约束方程，求解相机位姿变化

视觉里程计跟踪失败的情况下需要重定位进行恢复

视觉SLAM框架



视觉SLAM框架

后端

地图初始化

定义一个全局坐标系，一般以第一帧图像的相机坐标系作为全局坐标系

跟踪

位姿信息来自前端，后端记录特征点或者像素点对应的空间点的坐标

地图扩展

对当前跟踪成功的帧进行重建，将相关信息加入到已创建的地图中

闭环检测

与已创建的地图进行比较，判断相机是否运动到之前访问过的区域

地图优化

对已创建的地图进行优化，减小中的累积误差，获得准确的位姿和全局一致的地图

章节安排



什么是SLAM



SLAM发展简史



视觉SLAM算法



SLAM技术的基础理论



ROS下的SLAM实现

SLAM基础理论-多传感器融合

经典卡尔曼滤波的核心思想是递推求解，将SLAM建模为一个马尔科夫过程，每次只根据上一次的状态递推当前状态。将上面介绍的运动概率模型建模为线性模型的采样概率：

$$\mathbf{R}_k = \mathbf{A}\mathbf{R}_{k-1} + \mathbf{u}_{k-1} + \mathbf{q}_{k-1}$$

其中A为机器人姿态转移矩阵，描述了在没有运动控制指令输入情况下机器人位姿变化的规律。 $\mathbf{q}_{k-1}$ 为系统噪声，假设为高斯白噪声，具有固定的协方差矩阵Q。观测概率模型建模为对式20-4所示线性模型的采样概率：

$$\mathbf{z}_k = \mathbf{H}[\mathbf{R}_k, f(\mathbf{M}_k)]^T + \mathbf{r}_k$$

其中，H为观测矩阵， $\mathbf{r}_k$ 为观测噪声，假设为高斯白噪声，具有固定的协方差矩阵R

SLAM基础理论-多传感器融合

步骤1：给定机器人和地图的初始状态和协方差矩阵 $\mathbf{x}_0$ 、 $\mathbf{P}_0$

步骤2：状态预测与协方差推算

$$\begin{aligned}\tilde{\mathbf{R}}_k &= \mathbf{A}\mathbf{R}_{k-1} + \mathbf{u}_{k-1} \\ \tilde{\mathbf{P}}_k &= \mathbf{A}\mathbf{P}_{k-1}\mathbf{A}^T + \mathbf{Q}\end{aligned}$$

步骤3：计算卡尔曼增益

$$\mathbf{K}_k = \tilde{\mathbf{P}}_k \mathbf{H}^T (\mathbf{H}\tilde{\mathbf{P}}_k \mathbf{H}^T + \mathbf{R})^{-1}$$

步骤4：计算误差

$$\mathbf{R}_k = \tilde{\mathbf{R}}_k + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H}[\tilde{\mathbf{R}}_k, f(\mathbf{M}_k)]^T)$$

步骤5：更新误差协方差

$$\mathbf{P}_k = \tilde{\mathbf{P}}_k - \mathbf{K}_k \mathbf{H}\tilde{\mathbf{P}}_k$$

步骤6：增量式构建地图

$$\mathbf{M}_k = h(\mathbf{M}_{k-1}, \mathbf{x}_k, \mathbf{z}_k)$$

相机模型与视觉里程计

SLAM基础理论-后端优化

1. 滤波器后端
2. BA与图优化
3. Pose Graph

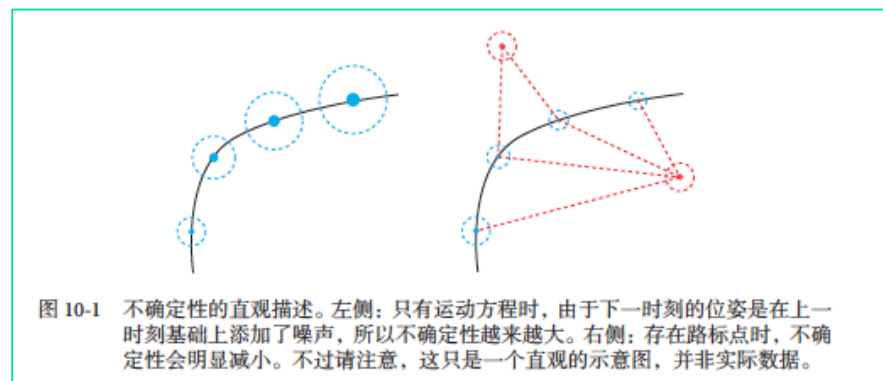
1. 滤波器后端

- **后端 (Backend)**
 - 从带噪声的数据估计内在状态——状态估计问题
 - Estimated the inner state from noisy data
- **渐进式 (Incremental)**
 - 保持当前状态的估计，在加入新信息时，更新已有的估计（滤波）
 - 线性系统+高斯噪声=卡尔曼滤波器
 - 非线性系统+高斯噪声+线性近似=扩展卡尔曼
 - 非线性系统+非高斯噪声+非参数化=粒子滤波器
 - Sliding window filter & multiple state Kalman (MSCKF)
- **批量式 (Batch)**
 - 给定一定规模的数据，计算该数据下的最优估计（优化）

1. 滤波器后端

• 简单的例子

- 当我们蒙着眼走路时：
 - 一开始我们知道自己在哪
 - 能粗略估计每步的距离，但不确定
 - 不确定性随时间累积
 - 导致自身的位置不确定性越来越大
- 某个时刻睁开眼睛时
 - 能够观测到周围
 - 尽管也有不确定性，但比估计步长小很多
 - 能够将不确定性保持在一定范围内



1. 滤波器后端

- 数学描述

- 符号定义:

- $t = 0 \dots N$ 时间内, 机器人的位姿为: x_0 到 x_N , 同时有路标 $y_1, \dots, y_M$

- 运动和观测方程记为:

$$\begin{cases} x_k = f(x_{k-1}, u_k) + w_k \\ z_{k,j} = h(y_j, x_k) + v_{k,j} \end{cases} \quad k = 1, \dots, N, j = 1, \dots, M.$$

- 注意: 运动和观测都受噪声影响; 可以没有运动方程 (纯视觉SLAM)

- 我们从贝叶斯滤波器方法来推导卡尔曼滤波

1. 滤波器后端

- 用随机变量表示状态，估计其概率分布
- k 时刻所有待估计的量组成 k 时刻状态： $x_k \triangleq \{x_k, y_1, \dots, y_m\}$.
- k 时刻观测统一记成： z_k 方程简化为：符号稍微有点冲突

$$\begin{cases} x_k = f(x_{k-1}, u_k) + w_k \\ z_k = h(x_k) + v_k \end{cases} \quad k = 1, \dots, N.$$

1. 滤波器后端

注意先验部分的组成:

k-1时刻的状态估计

$$P(x_k | x_0, u_{1:k}, z_{1:k-1}) = \int P(x_k | x_{k-1}, x_0, u_{1:k}, z_{1:k-1}) P(x_{k-1} | x_0, u_{1:k}, z_{1:k-1}) dx_{k-1}. \quad (10.6)$$

k时刻受先前状态的影响

分歧:

可以假设k时刻状态只和k-1时刻有关 (假设了一阶马尔可夫性)
或假设k时刻状态与先前所有时刻均有关 (不假设马尔可夫性)

1. 滤波器后端

- 假设一阶马尔可夫性的情况: $P(x_k | x_{k-1}, x_0, u_{1:k}, z_{1:k-1}) = P(x_k | x_{k-1}, u_k)$.
- 第二项: $P(x_{k-1} | x_0, u_{1:k}, z_{1:k-1}) = P(x_{k-1} | x_0, u_{1:k-1}, z_{1:k-1})$.
- 于是, 该公式指出了如何从k-1时刻的状态分布递推至k时刻的分布
 - 只是现在我们还没有代入具体的分布形式
- 在线性模型、高斯状态分布下, 我们将得到卡尔曼滤波器
 - 在非线性模型、高斯状态分布下, 可以在工作点附近线性展开, 得到扩展卡尔曼滤波器

1. 滤波器后端

- 卡尔曼滤波器的推导

- 线性模型和高斯噪声:

$$\begin{cases} x_k = A_k x_{k-1} + u_k + w_k \\ z_k = C_k x_k + v_k \end{cases} \quad k = 1, \dots, N.$$

$$w_k \sim N(0, R), \quad v_k \sim N(0, Q).$$

- 状态的高斯分布 (区别先后验) $P(x_{k-1}) = N(\hat{x}_{k-1}, \hat{P}_{k-1})$

后验

$$\bar{x}_k, \bar{P}_k$$

先验

1. 滤波器后端

• 预备知识：高斯分布的线性变换

• 假定 $x \sim N(m, S), y = Ax + b$ 则 y 亦服从高斯分布

• 有： $E[y] = E[Ax + b] = AE[x] + b = Am + b$

$$\begin{aligned} \text{Cov}[y] &= E[(y - E[y])(y - E[y])^T] \\ &= E[A(x - m)(x - m)^T A^T] = ASA^T \end{aligned}$$

1. 滤波器后端

预测 (k-1时刻后验通过运动方程推算k时刻先验)

$$P(x_k | x_0, u_{1:k}, z_{1:k-1}) = N(A_k \hat{x}_{k-1} + u_k, A_k \hat{P}_{k-1} A_k^T + R).$$

并记成:

根据观测方程, $\bar{x}_k = A_k \hat{x}_{k-1} + u_k$, $\bar{P}_k = A_k \hat{P}_{k-1} A_k^T + R$.

根据前面的Bayes展开:

$$P(z_k | x_k) = N(C_k x_k, Q).$$

需要计算两个分布的乘积

$$P(x_k | x_0, u_{1:k}, z_{1:k}) \propto P(z_k | x_k) P(x_k | x_0, u_{1:k}, z_{1:k-1}).$$

需要用到高斯
分布的线性组
合性质

先验

1. 滤波器后端

$$P(x_k | x_0, u_{1:k}, z_{1:k}) \propto P(z_k | x_k) P(x_k | x_0, u_{1:k}, z_{1:k-1}).$$

- 技巧：已经两侧均是高斯分布，所以： $N(\hat{x}_k, \hat{P}_k) = N(C_k x_k, Q) \cdot N(\bar{x}_k, \bar{P}_k)$.
- 既然都是高斯分布，我们比较指数部分的二次项和一次项部分即可。
- 指数部分展开：

$$(x_k - \hat{x}_k)^T \hat{P}_k^{-1} (x_k - \hat{x}_k) = (z_k - C_k x_k)^T Q^{-1} (z_k - C_k x_k) + (x_k - \bar{x}_k)^T \bar{P}_k^{-1} (x_k - \bar{x}_k).$$

- 比较 x_k 的二次和一次项系数，对于二次项，有： $\hat{P}_k^{-1} = C_k^T Q^{-1} C_k + \bar{P}_k^{-1}$.

1. 滤波器后端

- **再考虑一次项：** $(x_k - \hat{x}_k)^T \hat{P}_k^{-1} (x_k - \hat{x}_k) = (z_k - C_k x_k)^T Q^{-1} (z_k - C_k x_k) + (x_k - \bar{x}_k)^T \bar{P}_k^{-1} (x_k - \bar{x}_k).$
- **比较一次项系数：** $-2\hat{x}_k^T \hat{P}_k^{-1} x_k = -2z_k^T Q^{-1} C_k x_k - 2\bar{x}_k^T \bar{P}_k^{-1} x_k$
- **整理之：** $\hat{P}_k^{-1} \hat{x}_k = C_k^T Q^{-1} z_k + \bar{P}_k^{-1} \bar{x}_k$
- **两侧同乘 $\hat{P}_k$ 并定义：** $K = \hat{P}_k C_k^T Q^{-1}$, **得：**

$$\begin{aligned}\hat{x}_k &= \hat{P}_k C_k^T Q^{-1} z_k + \hat{P}_k \bar{P}_k^{-1} \bar{x}_k \\ &= K z_k + (I - K C_k) \bar{x}_k = \bar{x}_k + K (z_k - C_k \bar{x}_k).\end{aligned}$$

称为卡尔曼滤波的更新式

1. 滤波器后端

1. 预测:

$$\bar{x}_k = A_k \hat{x}_{k-1} + u_k, \quad \bar{P}_k = A_k \hat{P}_{k-1} A_k^T + R. \quad (10.24)$$

2. 更新: 先计算 K , 它又称为卡尔曼增益。

$$K = \bar{P}_k C_k^T (C_k \bar{P}_k C_k^T + R_k)^{-1}. \quad (10.25)$$

经典卡尔曼滤波器的五个公式给出了线性高斯系统的最优无偏估计

然后计算后验概率的分布。

$$\begin{aligned} \hat{x}_k &= \bar{x}_k + K (z_k - C_k \bar{x}_k) \\ \hat{P}_k &= (I - K C_k) \bar{P}_k. \end{aligned} \quad (10.26)$$

1. 滤波器后端

- Kalman Filter的非线性扩展: EKF

- 当 f, h 为非线性函数时:

$$\begin{cases} x_k = f(x_{k-1}, u_k) + w_k \\ z_k = h(x_k) + v_k \end{cases} \quad k = 1, \dots, N.$$

- 在工作点附近进行一阶Taylor展开:

$$x_k \approx f(\hat{x}_{k-1}, u_k) + \left. \frac{\partial f}{\partial x_{k-1}} \right|_{\hat{x}_{k-1}}^{\text{记F}} (x_{k-1} - \hat{x}_{k-1}) + w_k.$$

$$z_k \approx h(\bar{x}_k) + \left. \frac{\partial h}{\partial x_k} \right|_{\bar{x}_k} (x_k - \hat{x}_k) + n_k.$$

记H

1. 滤波器后端

- **预测部分:** $\bar{x}_k = f(\hat{x}_{k-1}, u_k), \quad \bar{P}_k = F \hat{P}_k F^T + R_k.$
- **卡尔曼增益:** $K_k = \bar{P}_k H^T (H \bar{P}_k H^T + Q_k)^{-1}.$
- **更新部分:** $\hat{x}_k = \bar{x}_k + K_k (z_k - h(\bar{x}_k)), \hat{P}_k = (I - K_k H) \bar{P}_k.$

1. 滤波器后端

• EKF优点

- 推导简单清楚，适用各种传感器形式
- 易于做多传感器融合

• EKF缺点

- 一阶马尔可夫性过于简单
- 可能会发散（要求数据不能有outlier）
- 线性化误差
- 需要存储所有状态量的均值和方差，平方增长

2. BA与图优化

2. BA与图优化

- Bundle Adjustment已经在之前介绍过了
 - 事实上BA属于批量式的优化方法
 - 给定很多个相机位姿与观测数据，计算最优的状态估计
 - 定义每个运动/观测方程的误差，并从初始估计开始寻找梯度下降

$$\frac{1}{2} \sum_{i=1}^m \sum_{j=1}^n \|e_{ij}\|^2 = \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^n \|z_{ij} - h(\xi_i, p_j)\|^2.$$

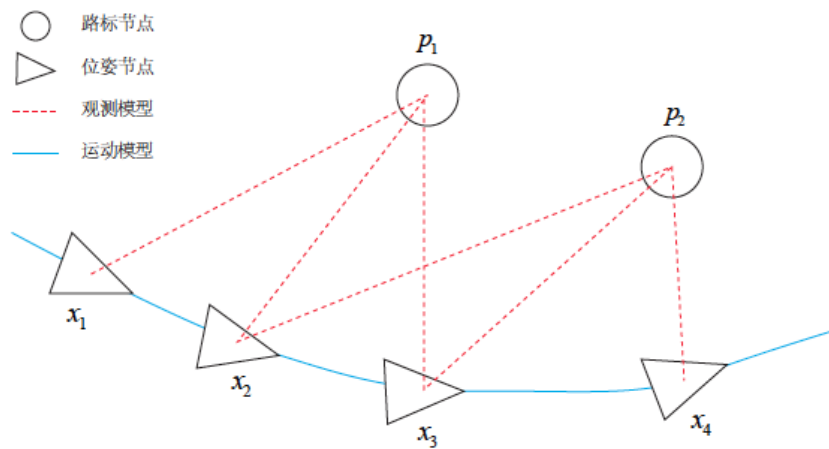


图 6-2 图优化的例子。

2. BA与图优化

• BA问题与图结构的关系

- BA虽然是个纯优化问题，但亦可以用图模型清晰地表述出来
- 顶点为优化变量，边为运动/观测约束
- 本身还有一些特殊的结构

• 考虑在位姿 i 处对路标 j 的一次观测 z_{ij} :

$$e_{ij} = z_{ij} - h(x_i, y_j)$$

特点:

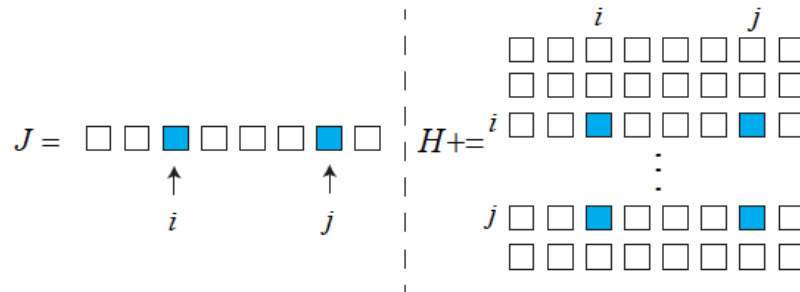
- 每个观测只关系两个变量，其中一个相机，一个是路标
- 纯视觉Ba中，不存在相机与相机/路标与路标之间的关联
- 整个误差函数由许多个这样小的项组成

2. BA与图优化

- 根据Gauss-Newton或Levenberg-Marquardt, 最终我们会求解 $Hx = -b$ 形式的方程
- 每一个误差项会对整体线程方程产生贡献:
- 而误差仅和 i, j 相关, 导致它的雅可比为稀疏矩阵: $H = \sum_{i,j} J_{ij}^T J_{ij}$

$$J_{ij}(x) = \left(0_{2 \times 6}, \dots, 0_{2 \times 6}, \frac{\partial e_{ij}}{\partial \xi_i}, 0_{2 \times 6}, \dots, 0_{2 \times 3}, \dots, 0_{2 \times 3}, \frac{\partial e_{ij}}{\partial p_j}, 0_{2 \times 3}, \dots, 0_{2 \times 3} \right).$$

- 示意图:

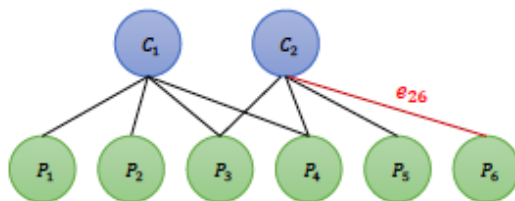


2. BA与图优化

- 如果对变量做好排序，例如所有相机位姿在前，路标在后，那么 H 有一定的特殊结构：

$$H = \begin{matrix} & \begin{matrix} C_1 & C_2 & P_1 & P_2 & P_3 & P_4 & P_5 & P_6 \end{matrix} \\ \begin{matrix} C_1 \\ C_2 \\ P_1 \\ P_2 \\ P_3 \\ P_4 \\ P_5 \\ P_6 \end{matrix} & \begin{bmatrix} \text{blue block} & & & & & & & \\ & \text{blue block} & & & & & & \\ & & \text{blue block} & & & & & \\ & & & \text{blue block} & & & & \\ & & & & \text{blue block} & & & \\ & & & & & \text{blue block} & & \\ & & & & & & \text{blue block} & \\ & & & & & & & \text{blue block} \end{bmatrix} \end{matrix}$$

Diagram illustrating the structure of the Hessian matrix H for Bundle Adjustment (BA). The matrix is partitioned by variables: camera poses (C_1, C_2) and landmarks ($P_1, P_2, P_3, P_4, P_5, P_6$). The matrix shows a block-diagonal structure for landmarks, with off-diagonal blocks representing the coupling between camera poses and the landmarks they observe. A red arrow points to the C_2 row and column, highlighting its connections to all observed landmarks.



图模型与H矩阵存在对应关系：

- 图模型中存在边 $\Rightarrow$ H 相应地方出现非零块

2. BA与图优化

- 实际当中的 H ，路标数量远大于位姿数量（箭头形矩阵或镐子形矩阵）

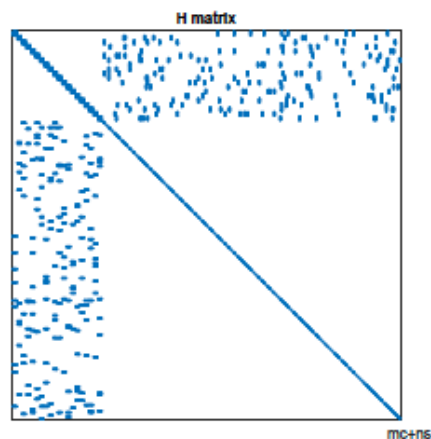
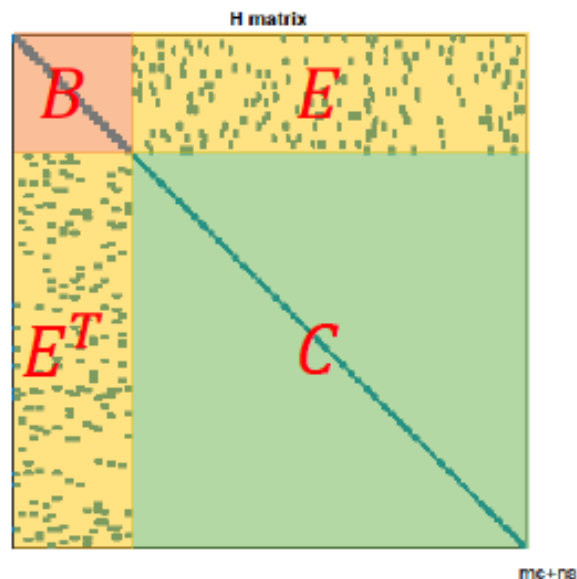


图 10-8 一般情况下的 H 矩阵。



2. BA与图优化

- 利用H的特殊结构，可以大幅加速 $Hx=-b$ 线性方程的求解
- 加速手段又称边缘化（Marginalization）[意义有很多种]
- $Hx=-b$ 的结构：

$$\begin{bmatrix} B & E \\ E^T & C \end{bmatrix} \begin{bmatrix} \Delta x_c \\ \Delta x_p \end{bmatrix} = \begin{bmatrix} v \\ w \end{bmatrix}.$$

B: 小, 对角块; C, 大, 对角块
E和E转置: 与图模型对应, 可稠密

- 因为 C 是对角块，所以可以用C对E进行消元（高斯消元），得到：

$$\begin{bmatrix} I & -EC^{-1} \\ 0 & I \end{bmatrix} \begin{bmatrix} B & E \\ E^T & C \end{bmatrix} \begin{bmatrix} \Delta x_c \\ \Delta x_p \end{bmatrix} = \begin{bmatrix} I & -EC^{-1} \\ 0 & I \end{bmatrix} \begin{bmatrix} v \\ w \end{bmatrix}. \quad \text{从而} \quad \begin{bmatrix} B - EC^{-1}E^T & 0 \\ E^T & C \end{bmatrix} \begin{bmatrix} \Delta x_c \\ \Delta x_p \end{bmatrix} = \begin{bmatrix} v - EC^{-1}w \\ w \end{bmatrix}.$$

2. BA与图优化

- 该方程组分为两步来求：

1. 求解上半部分，规模较小，得到
2. 将结果代入下半部分，得到

- 这个做法称为Marginalization或Schur消元

- 从消元角度来讲，亦可使用Cholesky等其他消元方式解此稀疏方程
- 从Marginalization角度来讲，是我们把所有的路标信息边缘化到了相机的信息中

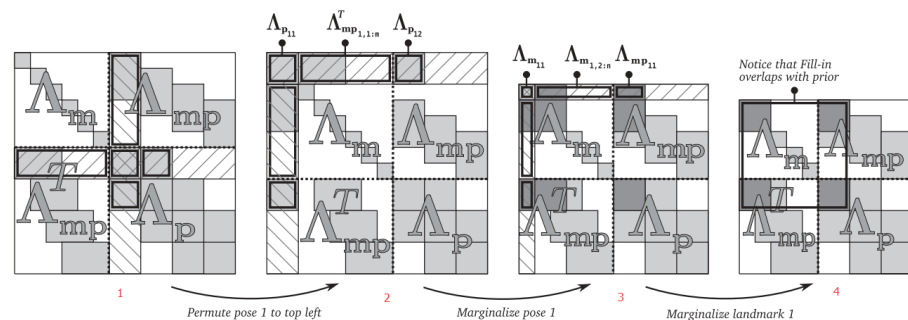
$$\begin{bmatrix} B - EC^{-1}E^T & 0 \\ E^T & C \end{bmatrix} \begin{bmatrix} \Delta x_c \\ \Delta x_p \end{bmatrix} = \begin{bmatrix} v - EC^{-1}w \\ w \end{bmatrix}.$$

2. BA与图优化

• Marginalization(marg)

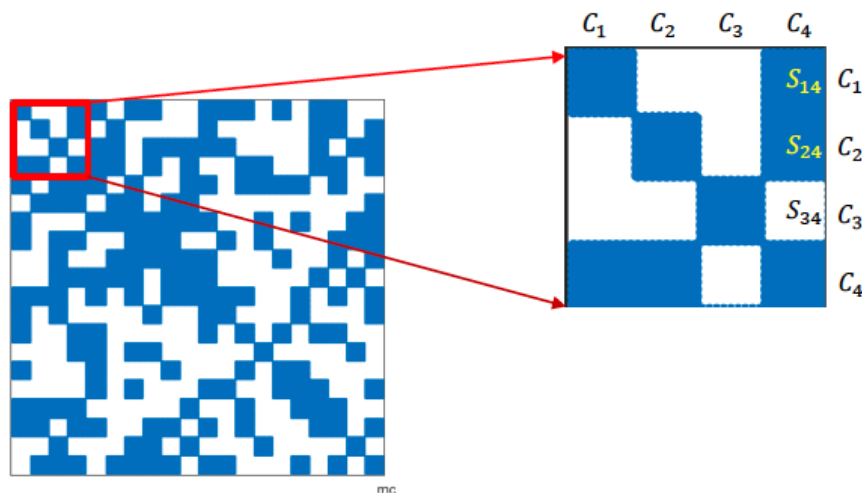
- 联合分布=边缘分布乘条件分布: $P(x_c, x_p) = P(x_c) \cdot P(x_p|x_c)$.
- 上述的做法marg掉了所有的路标点, 让概率全跑到相机位姿中, 这是为了加速求解 $Hx = -b$
- 然而, 在sliding window等其他做法中, 也可以选择边缘化一部分相机/路标点, 做法同上

- 但是, 通常情况下的Marg操作会给H矩阵带来填充 (fill-in), 使其不再具有稀疏结构;
- 所以, 要么刻意选择特定的marg策略, 要么仅使用稠密的H求解线性方程, 但这样效率会降低



2. BA与图优化

- Marginalization之后，左上角矩阵不再具有稀疏结构
- 但它反映了相机之间的共视关系 (Co-visibility)



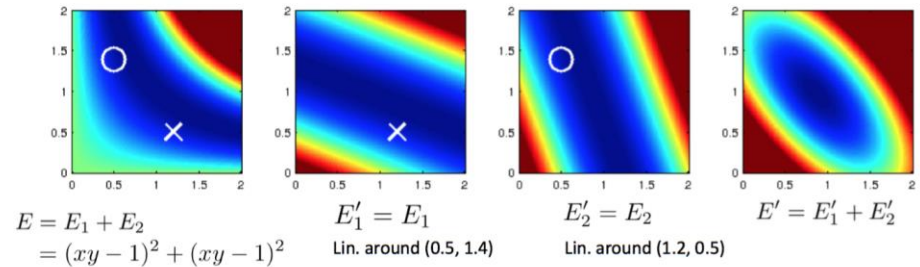
S14/S24处的非零块表示1,4相机, 2,4相机看到了同一个路标点

2. BA与图优化

- Marg操作与FEJ
- 在处理Marg时，被Marg的部分要使用First-estimate-Jacobian
- 否则可能导致整个问题零空间被降维

Windowed, real-time optimization: Consistency.

(for now, let's assume we have initializations, and know which points to use and where they are visible.)



nullspace disappears!

never combine linearizations around different linearization points,
 especially in the presence of non-linear nullspaces!
 It will render unobservable dimensions observable, and corrupt the system.

3. Pose Graph

3. Pose Graph

- 实际当中BA的计算量很大
 - 通常放在单独的后台线程中计算而无法实时
 - 主要计算来自于大量的特征点
- Pose Graph即是省略了特征点的Bundle Adjustment

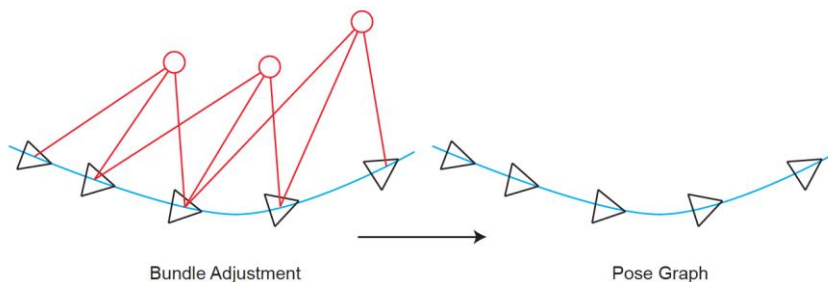


图 11-1 Pose Graph 示意图。当我们不再优化 Bundle Adjustment 中的路标点，仅把它们看成对姿态节点的约束时，就得到了一个计算规模减小很多的 Pose Graph。

3. Pose Graph

- **Pose Graph**

- 顶点仅由相机位姿组成
- 边为位姿与位姿间的约束

$$\Delta \xi_{ij} = \xi_i^{-1} \circ \xi_j = \ln \left(\exp \left((-\xi_i)^\wedge \right) \exp \left(\xi_j^\wedge \right) \right)^\vee,$$

$$\Delta T_{ij} = T_i^{-1} T_j.$$

- 不同的观测组成了不同的边
- 误差项:

$$\begin{aligned} e_{ij} &= \ln \left(\Delta T_{ij}^{-1} T_i^{-1} T_j \right)^\vee \\ &= \ln \left(\exp \left((-\xi_{ij})^\wedge \right) \exp \left((-\xi_i)^\wedge \right) \exp \left(\xi_j^\wedge \right) \right)^\vee. \end{aligned}$$

3. Pose Graph

• 误差项关于两个顶点的雅可比:

伴随

$$\exp((\text{Ad}(T)\xi)^\wedge) = T \exp(\xi^\wedge) T^{-1}.$$

$$\begin{aligned} \hat{e}_{ij} &= \ln(T_{ij}^{-1} T_i^{-1} \exp((- \delta \xi_i)^\wedge) \exp(\delta \xi_j^\wedge) T_j)^\vee \\ &= \ln(T_{ij}^{-1} T_i^{-1} T_j \exp((- \text{Ad}(T_j^{-1}) \delta \xi_i)^\wedge) \exp((\text{Ad}(T_j^{-1}) \delta \xi_j)^\wedge))^\vee \\ &\approx \ln(T_{ij}^{-1} T_i^{-1} T_j [I - (\text{Ad}(T_j^{-1}) \delta \xi_i)^\wedge + (\text{Ad}(T_j^{-1}) \delta \xi_j)^\wedge])^\vee \\ &\approx e_{ij} + \frac{\partial e_{ij}}{\partial \delta \xi_i} \delta \xi_i + \frac{\partial e_{ij}}{\partial \delta \xi_j} \delta \xi_j \end{aligned}$$

$$\frac{\partial e_{ij}}{\partial \delta \xi_i} = -\mathcal{J}_r^{-1}(e_{ij}) \text{Ad}(T_j^{-1}).$$

$$\frac{\partial e_{ij}}{\partial \delta \xi_j} = \mathcal{J}_r^{-1}(e_{ij}) \text{Ad}(T_j^{-1}).$$

$$\mathcal{J}_r^{-1}(e_{ij}) \approx I + \frac{1}{2} \begin{bmatrix} \phi_e^\wedge & \rho_e^\wedge \\ 0 & \phi_e^\wedge \end{bmatrix}.$$

SLAM基础理论-回环检测与建图



- 1. 回环检测与词袋**
- 2. 建图**
- 3. 展望**

1. 回环检测与词袋

1. 回环检测与词袋

- **回环检测的意义**

- VO和后端都存在误差
- SLAM的建图与定位是耦合的——误差将会累计

- **Loop Closing步骤**

- 检测到回环的发生
- 计算回环修正帧与当前帧的运动
- 验证回环是否成立
- 闭环

1. 回环检测与词袋

- 如何检测回环是否发生——回环检测
- 最简单方法：对任意两个关键帧进行特征匹配
- 基于里程计的方法 (Odometry based)
- 基于外观的方法 (Appearance based)
 - 外观方法是主流
 - 核心在于衡量图像间的相似性
 - 朴素的想法：灰度相减 $s(A, B) = \|A - B\|$.
 - 显然不好，不好在哪里？

1. 回环检测与词袋

表 12-1 回环检测的结果分类

算法 \ 事实	是回环	不是回环
是回环	真阳性 (True Positive)	假阳性 (False Positive)
不是回环	假阴性 (False Negative)	真阴性 (True Negative)

• 回环检测的指标

- **Perceptual Aliasing**
- **Perceptual Variability**

• ROC曲线 (Precision-Recall曲线)

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}), \quad \text{Recall} = \text{TP} / (\text{TP} + \text{FN}). \quad (12.2)$$

Precision和Recall通常是矛盾的
P高则R低
R高则P低



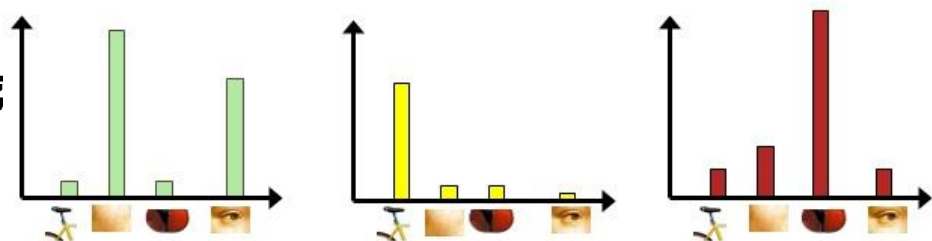
False Positive



False Negative

1. 回环检测与词袋

- Appearance-based主要方法：词袋
- Bag-of-Words, BoW
- BoW是对特征的聚类
 - 特征聚类形成了Word
 - 许多Word组成了Dictionary
 - 图像的相似性=Word的相似性
 - 只看Word的有无，无视Word的顺序



1. 回环检测与词袋

• Word的形成

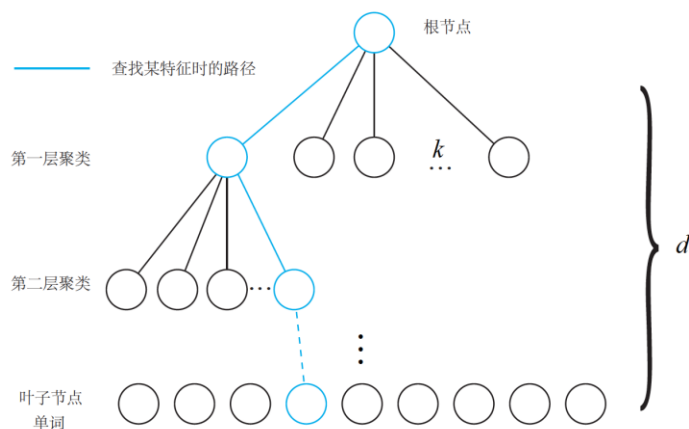
- 聚类
- 经典的k-means聚类 N 个特征点 $\rightarrow k$ 个类

1. 随机选取 k 个中心点: $c_1, \dots, c_k$ 。
2. 对每一个样本, 计算它与每个中心点之间的距离, 取最小的作为它的归类。
3. 重新计算每个类的中心点。
4. 如果每个中心点都变化很小, 则算法收敛, 退出; 否则返回第 2 步。

1. 回环检测与词袋

• 字典的结构

- 当实际拿到一个特征时，需要查询它对应的Word是什么
- 遍历比较： $O(n)$
- 建立字典结构可以加速比较



- k-d tree: k分枝树, d层
- k^d 个Word, d次比较, $O(kd)$

1. 回环检测与词袋

• 相似度计算

- 原则上比较Word即可计算图像相似度
- 但一些Word很常见, 另一些则很罕见
- TF-IDF (Term Frequency-Inverse Document Frequency)
 - 思路: 单词在字典中出现频率越高, 则区分度越低/在图像中频率越高-则区分度越高
 - IDF部分可在字典训练过程中计算
 - TF部分则需要对图像的特征进行计算
- 相似度:

$$A = \{(w_1, \eta_1), (w_2, \eta_2), \dots, (w_N, \eta_N)\} \triangleq \mathbf{v}_A.$$

$$s(\mathbf{v}_A - \mathbf{v}_B) = 2 \sum_{i=1}^N |\mathbf{v}_{Ai}| + |\mathbf{v}_{Bi}| - |\mathbf{v}_{Ai} - \mathbf{v}_{Bi}|.$$

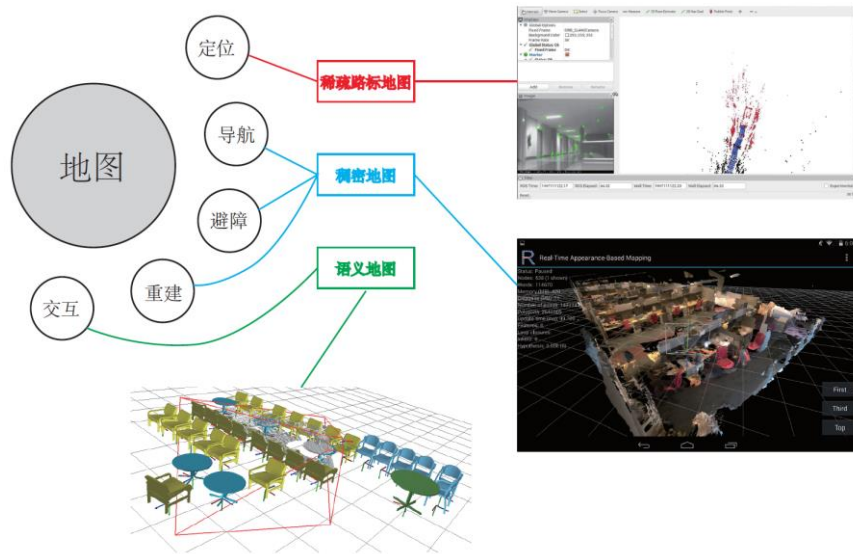
1. 回环检测与词袋

- **SLAM中的回环检测**
 - 使用相对的评分而非绝对的评分
 - 思路：相邻关键帧是相似的，而回环的相似度应该约等于相邻关键帧
- **检测成功以后：**
 - 根据PnP等算法计算运动关系
 - 根据重投影关系验证回环是否成立
 - 利用全局BA或Pose Graph进行优化

2. 建图

2. 建图

- 目前我们谈到的都是特征点地图
- 实际当中地图往往有不同形式，对应不同需求



- 定位仅需匹配特征点
- 导航和避障需要稠密障碍物信息
- 交互需要稠密的物体表面信息
- 高层任务需要语义信息

特征点地图仅满足很小一部分需求

2. 建图

• SLAM中的地图

- 等价于地图点的位置估计
- 单目SLAM中可看成地图点的深度估计
- 批量式 => Bundle Adjustment
- 递归式 => 极线搜索、深度滤波器

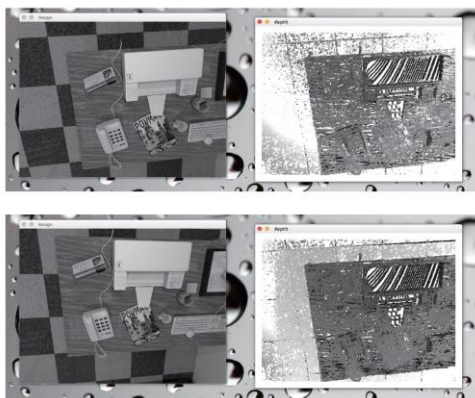


图 13-6 演示程序运行时截图。两图分别是迭代 10 次和 30 次的结果。

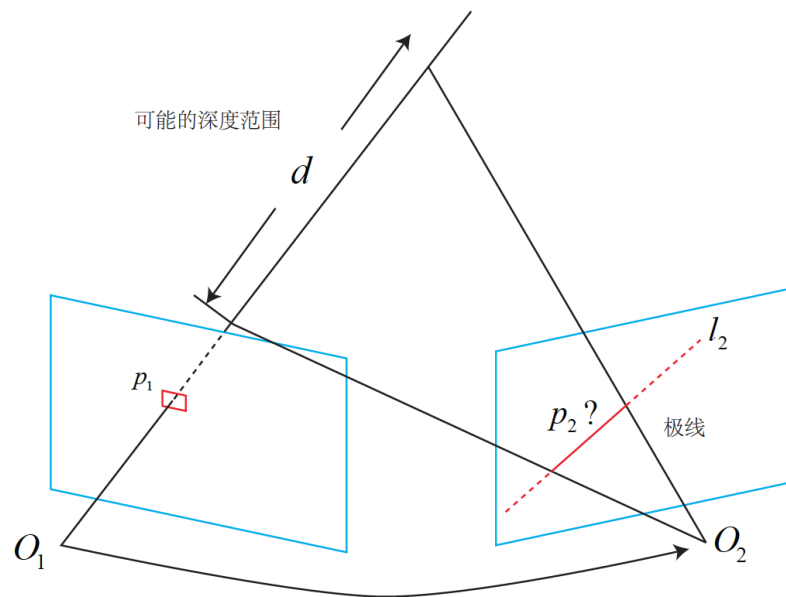
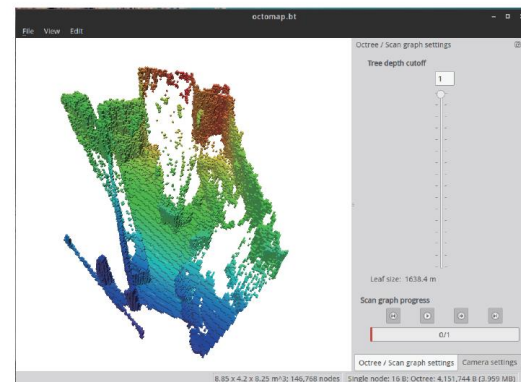


图 13-2 极线搜索示意图。

2. 建图

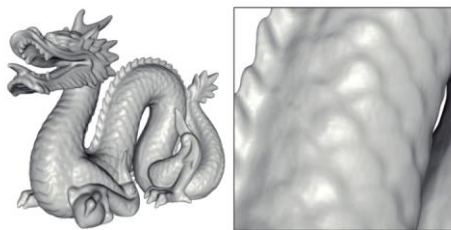
- RGBD稠密建图

- 点云
- 网格/面片
- TSDF
- 八叉树



八叉树地图 (0.05米分辨率)

泊松重建：从点云到网格 Surfel：以面片为单位的建图



泊松重建示例



Surfel重建示例

2. 建图

- **地图的表达仍在研究中**
 - 每一个地图展开谈都是比较大的主题
 - 动态/带人物/长时间的地图
 - 参数化的紧凑表达
 - 语义信息

3. 展望

- **SLAM已有众多的开源方案**
 - 开源能够让用户了解其原理, 促进研究的发展
- **今后的方向**
 - 多传感器融合 (V+IMU, V+GPS, V+ODOM)
 - **SLAM+DL**
 - Depth learning
 - Stereo learning
 - Descriptor Learning
 - Semantic mapping/BA
 - Etc.

表 14-1 常用开源 SLAM 方案

方案名称	传感器形式	地址
MonoSLAM	单目	https://github.com/hanmekim/SceneLib2
PTAM	单目	http://www.robots.ox.ac.uk/~gk/PTAM/
ORB-SLAM	单目为主	http://webdiis.unizar.es/~raulmur/orbslam/
LSD-SLAM	单目为主	http://vision.in.tum.de/research/vslam/lsdslam
SVO	单目	https://github.com/uzh-rpg/rpg_svo
DTAM	RGB-D	https://github.com/anuranbaka/OpenDTAM
DVO	RGB-D	https://github.com/tum-vision/dvo_slam
DSO	单目	https://github.com/JakobEngel/dso
RTAB-MAP	双目/RGB-D	https://github.com/introlab/rtabmap
RGBD-SLAM-V2	RGB-D	https://github.com/felixendres/rgbdslam_v2
Elastic Fusion	RGB-D	https://github.com/mp3guy/ElasticFusion
Hector SLAM	激光	http://wiki.ros.org/hector_slam
GMapping	激光	http://wiki.ros.org/gmapping
OKVIS	多目 +IMU	https://github.com/ethz-asl/okvis
ROVIO	单目 +IMU	https://github.com/ethz-asl/rovio

章节安排



什么是SLAM



SLAM发展简史



视觉SLAM算法



SLAM技术的基础理论



ROS下的SLAM实现

ROS简介

- ROS是一个先进的机器人操作系统框架，现今已有数百个研究团体和公司将其应用在机器人技术产业中。在2013年麻省理工学院科技评论（MIT Technology Review）中更指出：“从2010年发布1.0版本以来，ROS已经成为机器人软件的事实标准”。



定义

- ROS的官方定义为面向机器人的开源的元操作系统（meta-operating system）。
- 它能够提供类似传统操作系统的诸多功能，如硬件抽象、底层设备控制、常用功能实现、进程间消息传递和程序包管理等。

设计目标

- 设计目标：在机器人研发领域提高代码复用率。
- 途径：
分布式处理框架（又名Nodes）

多语言支持

- ROS被设计成了语言中立性的框架结构。ROS支持许多不同的语言，例如C++、Python、Octave和LISP，也包含其他语言的多种接口实现。
- 主要的库：
 - Python
 - C++
 - Lisp
- 试验库：
 - Java (with Android support)
 - Lua

精简与集成

- ROS建立的系统具有模块化的特点，各模块中的代码可以单独编译，而且编译使用的CMake工具使它很容易的就实现精简的理念。ROS基本将复杂的代码封装在库里，只是创建了一些小的应用程序为ROS显示库的功能，就允许了对简单的代码超越原型进行移植和重新使用。作为一种新加入的有优势，单元测试当代码在库中分散后也变得非常的容易，一个单独的测试程序可以测试库中很多的特点。

工具包丰富

- 为了管理复杂的ROS软件框架，大量的小工具被利用去编译和运行多种多样的ROS组件，从而设计成了内核，而不是构建一个庞大的开发和运行环境。
- 这些工具担任了各种各样的任务，例如，组织源代码的结构，获取和设置配置参数，形象化端对端的拓扑连接，测量频带使用宽度，生动的描绘信息数据，自动生成文档等等。

免费并且开源

- ROS所有的源代码都是公开发布的。这将必定促进ROS软件各层次的调试，不断的改正错误。

ROS工具

- RViz
- gazebo
- Tf

ROS工具

■ 1.RViz

- RViz是ROS中一款强大的3D可视化工具，机器人可以在该可视化工具中被创建，并让机器人完成指定的任务，还可以创建地图，显示3D点云等等。总之，RViz可以在ROS中显示所有的实物，通过消息的订阅可以完成实物的显示，机器人通过ROS发布数据，rviz订阅消息接收数据，然后显示。
- 详细的教程可以参考wiki：
- <http://www.ros.org/wiki/rviz>

ROS工具

■ 2.gazebo

- 这个工具是ROS中的物理仿真环境，gazebo本身就是一款机器人的仿真软件，基于ODE的物理引擎，可以模拟机器人以及环境中的很多物理特性。
- 详细的教程可以参考wiki：
- http://www.ros.org/wiki/simulator_gazebo/Tutorials

ROS工具

■ 3.Tf

- **tf**是ROS中的坐标变换系统，在机器人的建模仿真中经常用到。**tf**库的目的是实现系统中任一个点在所有坐标系之间的坐标变换，也就是说，只要给定一个坐标系下的一个点的坐标，就能获得这个点在其他坐标系的坐标。
- ROS中主要有两种坐标系：
 - （1）固定坐标系：用于表示世界的参考坐标系；
 - （2）目标坐标系：相对于摄像机视角的参考坐标系。
- 详细的教程可以参考wiki：
- <http://www.ros.org/wiki/tf>

SLAM

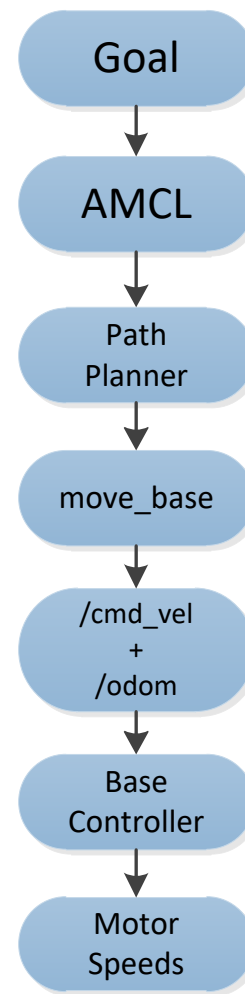
- 1.控制移动底座
- 2.导航、路径规划和SLAM

1.控制移动底座

■ 运动控制的分层

- 控制一个机器人运动可以在运动控制系统中许多不同的层中实现，ROS在不同的层中都有提供相应的方法。从对发动机的直接控制，到路径规划和SLAM（Simultaneous Localization and Mapping，实时定位和地图绘制），不同的层代表不同程度的抽象。
- 可将其分为六层
 - (1)发动机、轮子和编码器
 - (2)发动机控制器和驱动程序
 - (3)ROS基控制器
 - (4)使用ROS中move_base包的基于框架的运动
 - (5)使用ROS的gmapping包和amcl包的SLAM
 - (6)语义目标

- 运动控制的层次如图所示



2.导航、路径规划和SLAM

- 一个支持SLAM的机器人可以为一个未知的环境绘制地图，并实时地定位自己在地图上的位置。
- 直至最近，唯一可靠的SLAM方法就是用相当昂贵的激光扫描仪来收集数据。

2.导航、路径规划和SLAM

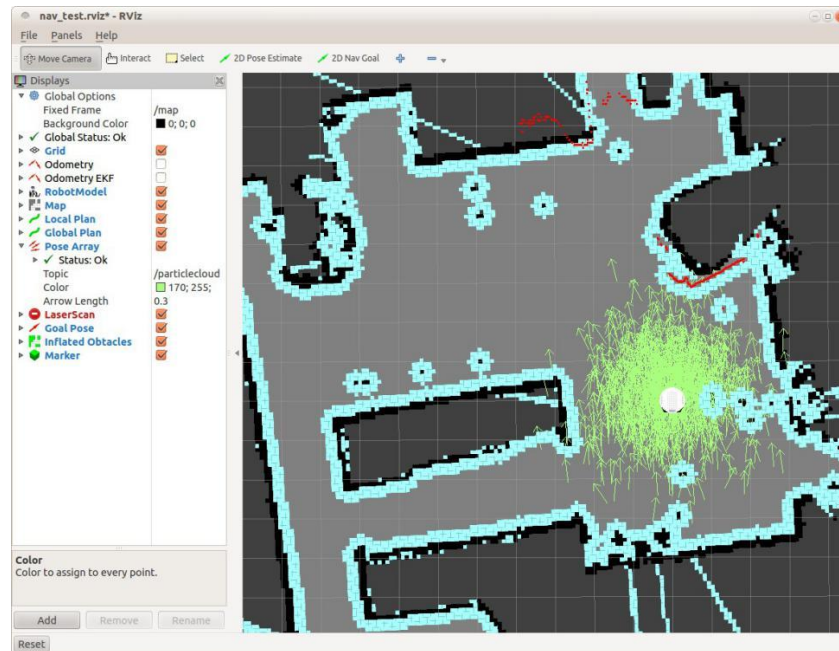
- 以下三个基础包（package）组成SLAM的核心
 - 使用move_base包进行路径规划和障碍物躲避
 - 用gmapping包创建地图
 - 用一张地图和amcl来导航和定位

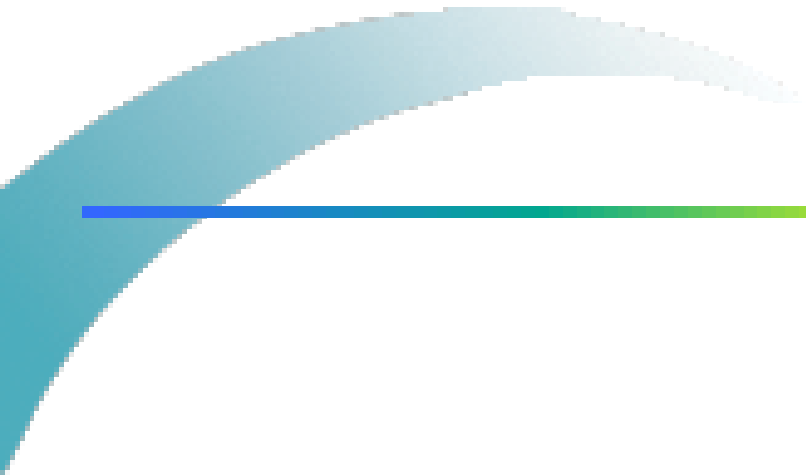
- **使用move_base包进行路径规划和障碍物躲避**
 - **Move_base包实现了一个完成指定导航目标的ROS行为。在机器人实现目标的过程中，行为是有反馈机制的。这意味着不再需要自己去通过odometry话题来判断是否已经达到目的地了。**
 - **move_base包（package）包含了base_local_planner，在为机器人寻路的时候，base_local_planner结合了从全局和本地地图得到的距离测量数据。**
 - **详见：
<http://wiki.ros.org/navigation/Tutorials/RobotSetup>**

- **用gmapping包创建地图**
 - ROS的gmapping包包含了slam_gmapping节点，这个节点会把从激光扫描仪和测量中得到的数据整合到一张occupancy map中。

- **用地图和amcl来导航和定位**
 - **ROS使用amcl包来让机器人在已有的地图里利用当前从机器人的激光或深度扫描仪中得到的数据进行定位。**
 - **amcl: adaptive Monte Carlo localization (蒙特卡洛定位方法的一种升级版,使用自适应的KLD方法来更新粒子)**

- 下图是一些现实测试中的截图：





谢 谢

